ML Foundations Crash Course

Problem, Data, and Objective Framing

Bryan Bischof

21 Jan 2026

Today's Journey

This lecture is a fast reset on core ML ideas, organized around three anchors:

Anchor	Core Question
Problem Framing	What decision are we trying to improve?
Data Framing	What evidence do we have (or can we get)?
Objective Framing	How will we measure and optimize "good"?

Everything else—models, training, metrics—is downstream of these three choices.

Why ML, in One Sentence

Machine learning is the practice of building systems that **infer** (predict, rank, classify, estimate) from data, and proving they work by **evaluating** those inferences against an objective on data that was not used to build the system.

The Essential Engineering Link

That "prove they work" part is the essential connection to the rest of this course: **evaluation as a test harness**.

Core Principle

If you can't evaluate it, you can't engineer it.

Outline

1. Problem Framing
2. Data Framing
3. Objective Framing
4. Optimization
5. Model Evaluation
6. A Tour of Models
7. A Tour of ML Paradigms
8. Statistics
9. Connecting to AI Engineering

What Is Problem Framing?

Problem framing is the translation from a vague desire ("make it smarter") into a well-scoped engineering goal.

A good problem framing answers:

Question	What It Clarifies
Decision	What decision will the system make or support?
Actionability	What changes if the prediction is different?
User + Workflow	Who consumes the output, when, and how?
Constraints	Latency, cost, privacy, interpretability, safety, compliance
Failure Costs	What is worse: false positives or false negatives?
Success Criteria	What does "better" look like in the real world?

Examples of Problem Framing

Spam Filtering (Classification)

- **Decision**: Route message to inbox vs. spam
- **Failure costs**: False positives are expensive (lost real mail)
- **Constraints**: Low latency; robust to adversarial text
- **Success**: Fewer spam at same (or lower) FP rate

Search (Ranking)

- **Decision**: Order documents so user finds the answer quickly
- **Failure costs**: Wrong top-1 is worse than wrong rank-50
- **Success**: Better top-k relevance and time-to-answer, not just accuracy

The ML Mindset vs. Traditional Programming

Traditional programming: Write explicit rules.

```
def classify_email(email: str) -> str:
    spam_words = ["lottery", "winner",
                  "click here", "urgent"]
    if any(w in email.lower()
           for w in spam_words):
        return "spam"
    return "not_spam"
```

This works until spammers adapt, edge cases multiply, and rules become unmaintainable.

ML: Provide examples, let the algorithm discover rules.

```
training_data = [
  ("Congratulations! You won...",
   "spam"),
  ("Meeting tomorrow at 3pm",
   "not_spam"),
  # ... thousands more
]
model = train(training_data)
pred  = model.predict(
    "Your package is delayed")
```

When to Use ML

Use ML when:

1. Rules are brittle
2. Patterns are complex or high-dimensional
3. You can define an evaluation method

Key Principle

If you can't evaluate it, you can't engineer it.

Types of ML Problems

Type	Output	Examples
Classification	Discrete categories	Spam detection, sentiment, image recognition
Regression	Continuous values	Price prediction, temperature forecasting
Ranking	Ordered list	Search results, recommendations
Generation	New content	Text, images, code

Outline

1 Problem Framing

2 Data Framing

3 Objective Framing

4 Optimization

5 Model Evaluation

6 A Tour of Models

7 A Tour of ML Paradigms

8 Statistics

9 Connecting to AI Engineering

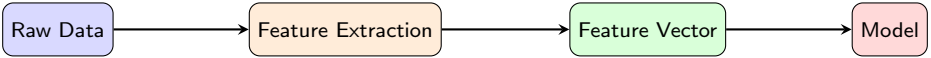
What Is Data Framing?

Data framing is about turning the problem into a dataset with a clear *unit of analysis* and a trustworthy measurement process.

Choice	**Question**
Unit of analysis	What is one row? A user? A session? A query? A document?
Inputs (features)	What signals are available at decision time?
Target (label)	What are we trying to predict (or approximate)?
Time	What timestamps matter? What is known at prediction time?
Sampling	Where does the data come from, and what gets left out?

Features and Representations

A **feature** is a measurable property of the data that might be useful for prediction.



Example — House Price Prediction:

```
features = {"square_feet": 1850, "bedrooms": 3,
  "bathrooms": 2, "year_built": 1985,
  "distance_to_downtown_miles": 5.2,
  "school_rating": 8}
```

Feature Engineering

Feature engineering is the art of creating useful features from raw data—often the difference between a mediocre model and a great one.

Technique	**Example**
Binning	Age $\rightarrow$ age_group (18–25, 26–35, . . .)
Interaction	bedrooms $\times$ bathrooms
Aggregation	Average transaction amount over last 30 days
Time features	Day of week, hour of day, is_weekend
Text features	Word count, sentiment score, TF-IDF
Encoding categories	One-hot encoding, target encoding

Data Is a Measurement Instrument

Treat your dataset like a sensor. Sensors have bias, noise, missingness, and failure modes.

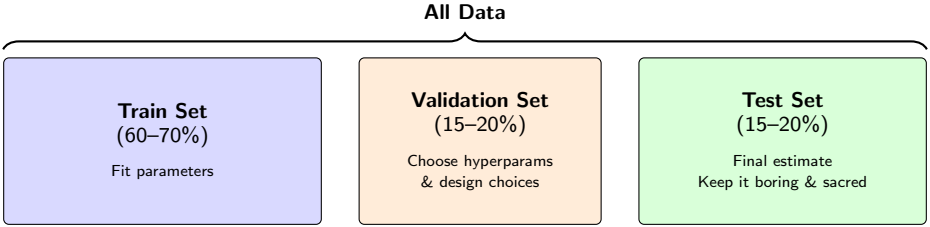
Issue	Description
Label noise	Human error, ambiguous cases, shifting definitions
Proxy labels	Clicks as proxy for satisfaction; purchases as proxy for "good recommendations"—useful and dangerous
Feedback loops	Your model changes user behavior, which changes data
Missing data	Systematic vs. random missingness
Outliers	Errors vs. real but rare phenomena
Class imbalance	99% not-fraud, 1% fraud

The Most Common Data Failure: Leakage

Data leakage: training data accidentally contains information unavailable at inference time. Inflates evaluation metrics; models fail in production.

Pattern	Example	Problem
Future information	Using tomorrow's stock price to predict today's	Won't have this at prediction time
Target leakage	Using "account_closed" to predict churn	Effect used to predict cause
Train-test contamination	Normalizing before splitting	Test statistics influence training
Duplicates across splits	Same user in train and test	Not testing generalization
Aggregates incl. label	Average outcome per group	Label information in features

Split Discipline



Critical rules:

- **Train**: Used to fit parameters
- **Validation**: Used to choose hyperparameters and make design decisions
- **Test**: Used *once* for a final estimate; keep it "boring and sacred"

Cross-Validation and Temporal Splits

K-Fold Cross-Validation (when data is limited):

Fold 1	Val	Train	Train	Train	Train
Fold 2	Train	Val	Train	Train	Train
Fold 3	Train	Train	Val	Train	Train
Fold 4	Train	Train	Train	Val	Train
Fold 5	Train	Train	Train	Train	Val

Temporal splits — if the world is time-dependent, prefer time-based splits:

```
# WRONG for time-series
X_train, X_test = train_test_split(data, random_state=42)

# RIGHT: Time-based split
train = data[data["date"] < "2024-01-01"]
test  = data[data["date"] >= "2024-01-01"]
```

Dataset Shift (Why Production Is Hard)

Type	What Changes	Example
Covariate shift	$P(X)$ changes	New users, new topics
Label shift	$P(Y)$ changes	Fraud rate changes
Concept drift	$P(Y \mid X)$ changes	Meaning of "spam" changes

Key Principle

Your evaluation dataset must resemble the distribution you will face.

Outline

1. Problem Framing
2. Data Framing
3. Objective Framing
4. Optimization
5. Model Evaluation
6. A Tour of Models
7. A Tour of ML Paradigms
8. Statistics
9. Connecting to AI Engineering

What Is Objective Framing?

Objective framing is how you decide what "good inference" means and how you will measure it. It turns engineering goals into metrics and tests.

The Key ML Idea to Carry Forward

1. **Objective framing** defines a test for inference quality (metric + protocol)
2. **Evaluation data** is the input to that test
3. You iterate on the system to improve the measured result

Loss Functions: Regression

A **loss function** measures how wrong the model's predictions are. Training is the process of finding parameters that minimize this loss.

Loss	Formula	Behavior
MSE	$\frac{1}{n}\sum(y-\hat{y})^2$	Heavily penalizes large errors; sensitive to outliers
MAE	$\frac{1}{n}\sum\lvert y-\hat{y}\rvert$	Linear penalty; more robust to outliers
Huber	MSE for small, MAE for large	Best of both worlds

Loss Functions: Classification and the Eval Gap

Classification losses:

Loss	Use Case
Cross-entropy (log loss)	Probabilistic classification; foundation of logistic regression
Hinge loss	SVMs; encourages confident predictions with margin

Training objective vs. evaluation metric:

Often, models optimize a convenient loss but you evaluate with a different metric:

- Train logistic regression with log loss; evaluate with F1 or PR-AUC
- Train a ranker with pairwise loss; evaluate with NDCG@k

Warning

If your loss and metric disagree, you can get "better training" but worse user outcomes. That is an **objective framing** problem, not a model problem.

The Key Insight: Loss Functions Define Behavior

What you optimize is what you get.

Example — medical diagnosis where:

- False negative (missing cancer) is catastrophic
- False positive (unnecessary biopsy) is inconvenient

Standard loss treats both errors equally. You might want asymmetric costs:

```
def weighted_loss(y_true, y_pred):
    fn_cost = 100  # Missing cancer is very bad
    fp_cost = 1    # Unnecessary test is inconvenient

    fn = ((y_true == 1) & (y_pred == 0)).sum()
    fp = ((y_true == 0) & (y_pred == 1)).sum()

    return fn_cost * fn + fp_cost * fp
```

Multi-Objective Reality

Most real systems are multi-objective:

- **Maximize utility**: accuracy, relevance, helpfulness
- **Minimize harm**: unsafe outputs, unfair outcomes
- **Minimize cost**: latency, tokens, compute, human review time

Objective framing turns this into something you can evaluate:

Component	Purpose
Primary metric	What you optimize for most
Guardrails	Metrics that must not regress (safety, latency, cost)
Slices	Metrics by subgroup, traffic type, or scenario

Goodhart's Law

If you only track a single metric, you will accidentally optimize the wrong thing.

Outline

1 Problem Framing

2 Data Framing

3 Objective Framing

4 Optimization

5 Model Evaluation

6 A Tour of Models

7 A Tour of ML Paradigms

8 Statistics

9 Connecting to AI Engineering

Why Training Works

Most training is: choose parameters θ that minimize a loss on training examples:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_\theta(x_i),\ y_i\big)$$

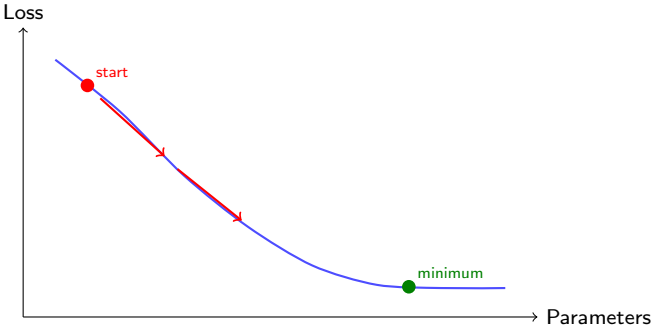
Where:

- f_θ: your model (parameterized by θ)
- ℓ: your training loss (objective function)

Gradient Descent

The workhorse of ML optimization:

1. Compute the gradient (direction of steepest increase) of the loss
2. Step *against* the gradient to reduce the loss
3. Repeat until improvements stop



Learning rate is crucial: too large → overshoot and diverge; too small → very slow convergence; just right → steady progress toward the minimum.

Stochastic Gradient Descent (SGD)

Computing gradients over all training data is expensive. SGD uses mini-batches:

```
for epoch in range(num_epochs):
    for batch in get_batches(training_data,
                             batch_size=32):
        gradients = compute_gradients(model, batch)
        model.parameters -= learning_rate * gradients
```

- Noisier gradients but much faster iterations
- Noise can help escape local minima
- Foundation of modern deep learning

Overfitting Is Optimization Doing Its Job Too Well

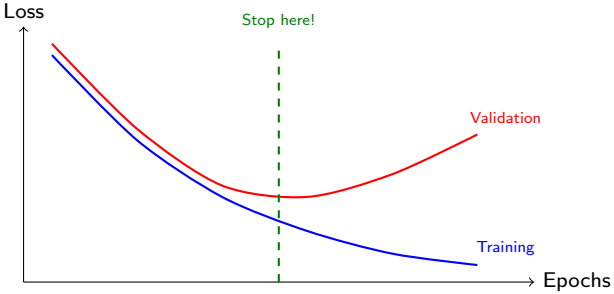
A model can fit the training data extremely well and still be wrong on new data:

```
Training accuracy: 99.9%
Test accuracy:     62.3%   <- Overfit!
```

That is why evaluation uses **held-out** data.

Regularization: Preventing Overfitting

Technique	How It Works
L2 (Ridge)	Penalize large weights: $L + \lambda \sum \theta_i^2$
L1 (Lasso)	Encourage sparse weights: $L + \lambda \sum \lvert\theta_i\rvert$
Dropout	Randomly zero out neurons during training
Early stopping	Stop training when validation loss starts increasing



Outline

1 Problem Framing

2 Data Framing

3 Objective Framing

4 Optimization

5 Model Evaluation

6 A Tour of Models

7 A Tour of ML Paradigms

8 Statistics

9 Connecting to AI Engineering

The Evaluation Paradigm

Evaluation answers: **how good is our inference on new data, under our objective?**

The ML Evaluation Loop

1. Define evaluation dataset (held-out, representative)
2. Pick metrics aligned with the objective
3. Run evaluation consistently (same data, code, slicing)
4. Use error analysis to guide iteration

The Confusion Matrix

	Predicted Neg	**Predicted Pos**
Actual Neg	TN	FP (Type I)
Actual Pos	FN (Type II)	TP

- **True Positives / True Negatives**: Correct predictions
- **False Positives (Type I)**: Incorrectly flagged as positive
- **False Negatives (Type II)**: Missed positives

Classification Metrics

Metric	Formula	When to Use
Accuracy	$\frac{TP+TN}{\text{Total}}$	Balanced classes, symmetric costs
Precision	$\frac{TP}{TP+FP}$	FP are costly (spam filter)
Recall	$\frac{TP}{TP+FN}$	FN are costly (disease screening)
F1	$2 \cdot \frac{P \cdot R}{P+R}$	Need single number, care about both
PR-AUC	Area under PR curve	Imbalanced classes, rare positives

Warning

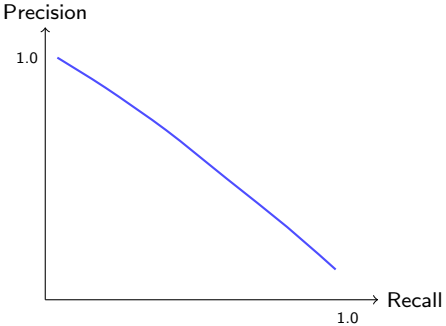
Accuracy is misleading for imbalanced classes! A model that always predicts "not fraud" achieves 99% accuracy when fraud is 1%.

Thresholds and Operating Points

Most classifiers output a probability. Your **objective framing** picks how to convert that into action:

```
prediction = "positive" if model.predict_proba(x) > threshold
             else "negative"
```

Adjusting the threshold trades off precision and recall:



Key Point

Regression Metrics

Metric	Formula	Interpretation
MAE	$\frac{1}{n}\sum\lvert y-\hat{y}\rvert$	Robust to outliers; interpretable units
MSE	$\frac{1}{n}\sum(y-\hat{y})^2$	Punishes large errors more
RMSE	$\sqrt{\text{MSE}}$	Same units as target
R^2	$1-\frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$	Variance explained (0–1)

Ranking / Retrieval Metrics

Metric	What It Measures
Precision@k	Fraction of top-k that are relevant
Recall@k	Fraction of relevant items in top-k
MRR	How soon does the first relevant item appear?
NDCG@k	Graded relevance with top-heavy weighting

Baseline Models

Always compare against baselines:

Type	Example	Purpose
Random	Random predictions	Sanity check
Majority class	Always predict most common	Bar for imbalanced data
Simple heuristic	Rule-based approach	Domain knowledge baseline
Simple model	Logistic regression	Complexity baseline

If your complex model doesn't beat simple baselines, something is wrong.

Error Analysis: The Fastest Path to Improvement

After you compute metrics, look at failures:

- Bucket errors by type (missing data, ambiguous labels, rare topics)
- Add slices that correspond to real product risk
- Use this to improve **data framing** (labels, coverage) and **objective framing** (metrics, rubrics)

The Improvement Loop

Metrics tell you *how well* the system performs. Error analysis tells you *why* it fails and *what to fix next*.

Outline

1. Problem Framing
2. Data Framing
3. Objective Framing
4. Optimization
5. Model Evaluation
6. A Tour of Models
7. A Tour of ML Paradigms
8. Statistics
9. Connecting to AI Engineering

Pick the Simplest Model That Meets Your Constraints

You can do a lot with a small set of model families.

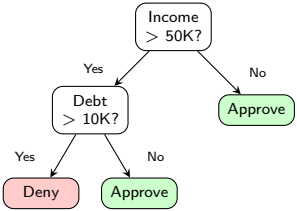
Linear Models (Logistic / Linear Regression):

$$P(y{=}1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

Aspect	Details
Strengths	Fast, stable, interpretable, hard to overfit with regularization
Failure mode	Underfits complex patterns unless features are engineered well
Use when	You need interpretability, have limited data, or as a baseline

Trees and Ensembles

Decision Trees: Sequence of if-then-else rules



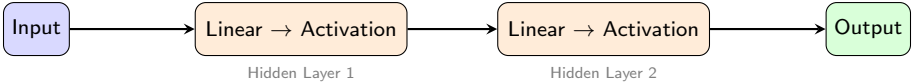
Ensembles:

- **Random Forests**: Ensemble of trees, each on random subset of data/features
- **Gradient Boosting** (XGBoost, LightGBM, CatBoost): Trees trained sequentially, each correcting prior errors

Strengths	Strong tabular baseline, handles non-linear interactions
Failure	Can overfit with leakage; harder to calibrate
Use when	Tabular/structured data

Neural Networks

Basic architecture:

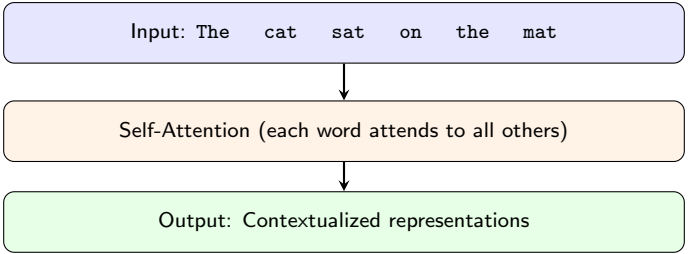


Aspect	Details
Strengths	Representation learning; great for text, vision, audio; scalable
Failure mode	Data hungry, sensitive to training setup, harder to debug
Use when	Unstructured data (text, images), large datasets

Transformers: The Foundation of Modern LLMs

The transformer architecture (Vaswani et al., 2017) is the basis for GPT, BERT, Claude, and all modern LLMs.

Key innovation: Self-Attention — each position can "attend to" all other positions:



Connection

We'll dive deeper in Unit 2. For now, understand that everything we've covered (loss functions, optimization, evaluation) applies to transformers too.

Outline

1. Problem Framing
2. Data Framing
3. Objective Framing
4. Optimization
5. Model Evaluation
6. A Tour of Models
7. A Tour of ML Paradigms
8. Statistics
9. Connecting to AI Engineering

Supervised and Unsupervised Learning

You don't need all the details today; you need to recognize which setting you're in because it determines what "data" and "evaluation" mean.

Supervised Learning

You have labeled pairs (x, y). Train to predict y from x.

- **Examples**: Spam detection, sentiment, credit risk, QA
- **Data need**: Labeled examples
- **Evaluation**: Compare predictions to held-out labels

Unsupervised Learning

No labels; learn structure in x.

- **Examples**: Clustering users, anomaly detection, dimensionality reduction
- **Data need**: Unlabeled examples
- **Evaluation**: Domain-specific metrics (cluster quality, reconstruction error)

Self-Supervised, Semi-Supervised, and Reinforcement Learning

Self-Supervised

Create "labels" from data itself.

- **Example**: Next-token prediction on text
- **Data**: Large text corpus (no manual labels)
- **Eval**: Perplexity, downstream tasks

How modern LLMs get built.

Semi-Supervised

Mix small labeled + large unlabeled data.

- **Example**: Pseudo-labeling, consistency regularization
- **Use when**: Labels are expensive but unlabeled data is plentiful

Reinforcement Learning

Learn a policy by feedback on actions.

- **Example**: RLHF / DPO for chatbots
- **Data**: Reward signal or preference comparisons
- **Eval**: Reward achieved, preference win rate

Outline

1 Problem Framing

2 Data Framing

3 Objective Framing

4 Optimization

5 Model Evaluation

6 A Tour of Models

7 A Tour of ML Paradigms

8 Statistics

9 Connecting to AI Engineering

Sampling and Uncertainty

Statistics matters in ML because evaluation produces numbers from samples. Your evaluation metric has **uncertainty**.

If you evaluate on n independent examples, your metric is an estimate. As n grows, the estimate becomes more stable.

Practical guidance:

- Report **both** the metric and the dataset size
- Prefer **confidence intervals** (or bootstraps) for key comparisons
- Watch out for cherry-picked slices; predefine slices when possible

Correlation Is Not Causation

Your dataset is an observation of a process you did not control. Confounding and selection bias can make models look good offline and fail online.

In AI systems, this shows up as:

- **Clicks as feedback**: Biased toward what you showed
- **Human ratings**: Inconsistent rubrics, drift over time
- **Self-fulfilling outputs**: Model changes what gets measured

Outline

1. Problem Framing
2. Data Framing
3. Objective Framing
4. Optimization
5. Model Evaluation
6. A Tour of Models
7. A Tour of ML Paradigms
8. Statistics
9. Connecting to AI Engineering

Classical ML → AI/LLM Systems

LLM outputs are often subjective, multi-dimensional, and non-deterministic. The same framing still applies:

Classical ML	**AI / LLM Systems**
Training set	Few-shot examples
Test set	Golden dataset (eval set)
Accuracy / F1	LLM-as-judge scores, rubric scores
Cross-validation	Multiple eval scenarios
Overfitting	Prompt overfitting
Data leakage	Eval contamination

The Practical Translation

Framing for AI systems:

- **Problem framing**: What decision or behavior are we specifying?
- **Data framing**: What are the test cases (prompts, contexts, tools, traces)?
- **Objective framing**: What rubric defines "good"? What are guardrails?

The implementation:

1. Build a **golden dataset** of representative scenarios
2. Define a **rubric** (possibly multi-metric)
3. Evaluate with **humans**, **LLM-as-judge**, or both
4. Track regressions in CI like unit tests

What Changes and What Stays the Same

Aspect	Classical ML	LLM-Based Systems
Training	You train the model	Model is pre-trained; you prompt/fine-tune
Features	You engineer features	Model handles raw text
Evaluation	Objective metrics	Often subjective ("is this summary good?")
Iteration	Retrain with new data	Adjust prompts, context, retrieval

What stays the same:

1. You need **evaluation data** — can't improve what you can't measure
2. You need **clear success criteria** — "make it better" isn't actionable
3. You need to **prevent contamination** — test data must be truly held out
4. You need **baselines** — how good is "good"?
5. You need to **iterate systematically** — change one thing at a time

The P^3 Framework: Promise → Proof → Production

The three anchors (problem, data, objective framing) map onto a lifecycle we call **P^3 — Promise, Proof, Production**. We'll use this framing throughout the course, and every AI system you build should pass through these three phases:

Promise

JTBD → testable hypothesis

- Job story: "When _, I want _, so I can _"
- Falsifiable acceptance criteria
- Safety "must-not" rules
- Scope boundaries
- Failure model: catastrophic vs. tolerable

Proof

Evals-first iteration loop

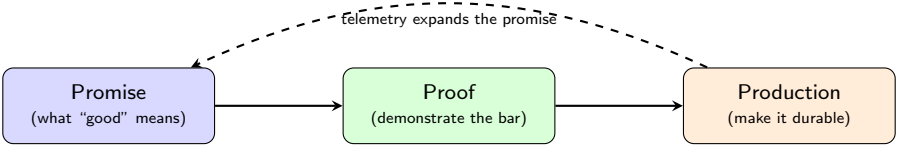
- Build measurement *before* changing anything
- Eval set: representative + long-tail + adversarial
- Scorable rubric + baseline
- Small hypothesis-driven experiments
- Pre-registered success/regression thresholds
- Error analysis → expand eval set

Production

Make proof durable

- Architecture based on proof gaps
- Observability for attribution
- Ship behind flags + regression gates
- Monitor drift and failures
- Telemetry feeds back into evals

P^3: Connecting the Anchors



Anchor	Phase	What you produce
Problem framing	Promise	Job story, acceptance criteria, failure model
Data + Objective framing	Proof	Eval set, rubric, baseline, deltas
Engineering discipline	Production	Flags, gates, observability, feedback loop

The Loop

Promise says what "good" means. **Proof** demonstrates the system meets that bar. **Production** ensures the proof holds under real-world conditions and keeps improving.

P^3 Across the Course

Unit	P^3 phase in focus
Unit 2 (LLMs)	**Promise** — knowing what an LLM can and can't deliver sets realistic acceptance criteria
Unit 3 (Evals)	**Proof** — the Analyze–Measure–Improve lifecycle is the machinery of proof
Unit 4 (Prompts)	**Promise** → **Proof** — a system prompt *is* a promise; prompt versioning and testing *is* proof
Unit 5 (Tools & MCP)	**Production** — architecture choices driven by proof gaps; observability enables proof
Unit 6 (Retrieval)	**Promise + Proof** — RAG's promise is grounded answers; retrieval quality is the proof bottleneck

Every subsequent unit will call back to P^3 when a concept maps onto promise, proof, or production. Watch for it.

Outline

1. Problem Framing
2. Data Framing
3. Objective Framing
4. Optimization
5. Model Evaluation
6. A Tour of Models
7. A Tour of ML Paradigms
8. Statistics
9. Connecting to AI Engineering

Key Takeaway

Objective framing creates the test, and the evaluation dataset is how you run it.

This is the central idea that connects classical ML to everything we'll do with LLMs, retrieval, and agents in this course.

Looking ahead:

- **Unit 2**: LLMs as a specific ML system with special constraints (tokens, sampling, cost, non-determinism)
- **Unit 3**: Deep dive on evaluation—golden datasets, rubrics, judges, and CI

The evaluation mindset we established today will guide how we think about LLM-based systems throughout the course.

Checklists (Keep These Handy)

Problem Framing

- ☐ Can I explain the decision in one sentence?
- ☐ Do I know who consumes the output and what they do with it?
- ☐ Do I know which error type is more costly?
- ☐ Have I identified constraints (latency, cost, safety)?

Data Framing

- ☐ Is my unit of analysis clear?
- ☐ Is my label definition stable and measurable?
- ☐ Does every feature exist at inference time?
- ☐ Split strategy matches deployment?
- ☐ Have I checked for leakage?

Objective Framing

- ☐ Does the metric match the decision and costs?
- ☐ Do I have guardrails and slices?
- ☐ Is my eval dataset representative?
- ☐ Have I defined baselines?

In-Class Exercise: Framing Worksheet (15–20 min)

Pick one system:

1. A course assistant that answers questions using course notes
2. A tool that ranks documents for a user query (RAG retrieval stage)
3. A classifier that flags unsafe or low-quality outputs

Problem Framing

- Decision:
- User/workflow:
- Failure costs:
- Constraints:

Data Framing

- Unit of analysis:
- Inputs at decision time:
- Label definition (or proxy):
- Leakage risks:
- Proposed split strategy:

Objective Framing

- Primary metric:
- Guardrails:
- Key slices:
- Evaluation dataset plan:

Litmus Test

If you can fill this out cleanly, implementation is usually straightforward.

Lab Preview

In today's lab, you'll:

1. Build a complete ML pipeline with proper train/test splits
2. Implement multiple evaluation metrics
3. Compare models and understand the precision-recall tradeoff
4. Experience how data leakage inflates metrics

This hands-on experience will make the concepts concrete and prepare you for building evaluation pipelines for LLM systems.

Thank you!